Computer Networks Lab Manual

Oct 29, 2026 · @Sneha Naik

All 14 practicals with simple programs and step-by-step Packet Tracer guides, ready to copy and run.

Before you start

You need two free tools: Cisco Packet Tracer (download from Cisco Networking Academy) and Python 3 (python.org). Every program below is plain Python with no extra installs.

Running the Python programs

1. Save the code in a file, for example `hamming.py`.
2. Open a terminal in that folder and run `python hamming.py`.
3. For client-server programs (7, 8, 9), open **two** terminals. Run the server first, then the client.

Packet Tracer basics used in every practical

- Pick devices from the bottom-left panel (End Devices, Network Devices). Drag them onto the workspace.
- Use the **Copper Straight-Through** cable between different device types (PC to switch, switch to router, server to switch).
- Use the **Copper Cross-Over** cable between same device types (switch to switch, router to router, PC to router).
- Click a PC, open the **Desktop** tab, then **IP Configuration** to set an address and **Command Prompt** to run `ping`.
- Wait about 30 seconds for the link lights to turn green before you ping.

Practical 1: Network topologies in Packet Tracer

Build each topology on its own part of the workspace and give each its own address range, so the four never mix.

A. Star topology (192.168.1.0/24)

1. Place one **2960 switch** and four **PCs**.
2. Connect each PC to the switch with a Straight-Through cable.
3. Set static IPs under Desktop > IP Configuration: PC0 `192.168.1.1`, PC1 `.2`, PC2 `.3`, PC3 `.4`, mask `255.255.255.0`.
4. From PC0 open Command Prompt and run `ping 192.168.1.2`, then `.3` and `.4`.

B. Mesh topology (192.168.2.0/24)

PCs have one port, so build the mesh between four switches, each with one PC.

1. Place four 2960 switches (SW1 to SW4) and one PC on each.
2. Join every switch to every other switch with Cross-Over cables: 6 links (1-2, 1-3, 1-4, 2-3, 2-4, 3-4).
3. Set PC IPs `192.168.2.1` to `192.168.2.4`, mask `255.255.255.0`.
4. Ping between all PCs. Pings work, though the switches will turn some links amber because Spanning Tree blocks loops. Mention this in your viva.

C. Bus topology (192.168.3.0/24)

Packet Tracer has no bus cable, so a chain of hubs acts as the shared bus.

1. Place four **Hubs** in a row and one PC on each hub.
2. Join hub to hub with Cross-Over cables (Hub1-Hub2, Hub2-Hub3, Hub3-Hub4). Join each PC to its hub with Straight-Through.
3. Set PC IPs `192.168.3.1` to `192.168.3.4`.
4. Ping from the first PC to the last PC.

D. Extended star topology (192.168.4.0/24)

1. Place one central switch (SW0) and three more switches (SW1, SW2, SW3).
2. Connect SW1, SW2, SW3 to SW0 with Cross-Over cables.
3. Place two PCs on each of SW1, SW2, SW3 with Straight-Through cables (6 PCs).
4. Set IPs `192.168.4.1` to `192.168.4.6`.
5. Ping a PC under SW1 from a PC under SW3.

Comparison

Topology	Cost	Reliability	Fault tolerance
Star	Medium: one cable per device plus a switch	Good: one cable failure affects one device	Low at the centre: if the switch fails, everything stops
Mesh	Very high: n(n-1)/2 links	Very high: many paths	Very high: traffic reroutes around a failed link
Bus	Lowest: one shared cable	Low: heavy traffic slows everyone	Very low: a break in the main cable splits the network
Extended star	Higher than star: extra switches and cables	Good: a failure stays within one branch	Medium: central switch is a single point of failure for branches

Practical 2: Hamming Code (7,4)

The sender adds 3 parity bits to 4 data bits. The receiver recalculates the parity checks; the result, read as a binary number, is the position of the wrong bit.

Bit positions 1 to 7 are: p1 p2 d1 p3 d2 d3 d4. Parity p1 checks positions 1,3,5,7; p2 checks 2,3,6,7; p3 checks 4,5,6,7.

```
# hamming.py
def encode(d):
    p1 = d[0] ^ d[1] ^ d[3]
    p2 = d[0] ^ d[2] ^ d[3]
    p3 = d[1] ^ d[2] ^ d[3]
    return [p1, p2, d[0], p3, d[1], d[2], d[3]]

def find_error(r):
    c1 = r[0] ^ r[2] ^ r[4] ^ r[6]
    c2 = r[1] ^ r[2] ^ r[5] ^ r[6]
    c3 = r[3] ^ r[4] ^ r[5] ^ r[6]
    return c3 * 4 + c2 * 2 + c1      # 0 means no error

bits = input("Enter 4 data bits (example 1011): ")
data = [int(b) for b in bits]
```

```
sent = encode(data)
print("Transmitted code :", sent)

pos = int(input("Error position to introduce (1-7, 0 for none): "))
received = sent[:]
if pos != 0:
    received[pos - 1] ^= 1           # flip one bit
print("Received code    :", received)

err = find_error(received)
if err == 0:
    print("No error detected")
else:
    print("Error detected at position", err)
    received[err - 1] ^= 1           # correct it
    print("Corrected code   :", received)

print("Original data    :", [received[2], received[4], received[5],
received[6]])
```

Sample run: data `1011` gives code `[0, 1, 1, 0, 0, 1, 1]`. Entering error position `5` flips bit 5, the receiver reports position 5, fixes it, and recovers `1011`.

Steps to explain in the viva

1. Encode: place data bits in positions 3, 5, 6, 7 and compute parity at positions 1, 2, 4.
2. Introduce a single-bit error by flipping one position.
3. Detect: recompute the three checks; the number they form is the error position.
4. Correct: flip that bit back.

Practical 3: Subnetting 192.168.1.0/24 into two subnets (Packet Tracer)

The router is set up entirely by clicking in its Config tab, so you do not type any commands.

Step 1: Calculate the subnet mask

1. We need 2 subnets. 2^1 = 2, so borrow **1 bit** from the host part.
2. The prefix goes from /24 to **/25**. The mask is `255.255.255.128`.

3. Host bits left: 32 - 25 = 7, so each subnet has 2^7 = 128 addresses and 128 - 2 = **126 usable hosts**.

	Network	Usable range	Broadcast	Router address (gateway)
Lab 1	192.168.1.0/25	.1 to .126	192.168.1.127	192.168.1.1
Lab 2	192.168.1.128/25	.129 to .254	192.168.1.255	192.168.1.129

Step 2: Build the network

1. Place one **2911 router**, two **2960 switches**, and four **PCs**.
2. Connect Router `GigabitEthernet0/0` to Switch0 and Router `GigabitEthernet0/1` to Switch1 with Straight-Through cables.
3. Connect PC0 and PC1 to Switch0 (Lab 1). Connect PC2 and PC3 to Switch1 (Lab 2).

Step 3: Configure the router by clicking

1. Click the router and open the **Config** tab.
2. In the left list, click **GigabitEthernet0/0**. Fill in IP Address `192.168.1.1` and Subnet Mask `255.255.255.128`, then tick **On** next to Port Status.
3. Click **GigabitEthernet0/1**. Fill in IP Address `192.168.1.129` and Subnet Mask `255.255.255.128`, then tick **On**.
4. Both router ports should now show green lights on the workspace. The router now routes between the two labs.

Step 4: Set the PC addresses

Click each PC, open **Desktop > IP Configuration**, choose **Static**, and enter:

PC	Lab	IP address	Subnet mask	Default gateway
PC0	Lab 1	192.168.1.2	255.255.255.128	192.168.1.1
PC1	Lab 1	192.168.1.3	255.255.255.128	192.168.1.1
PC2	Lab 2	192.168.1.130	255.255.255.128	192.168.1.129
PC3	Lab 2	192.168.1.131	255.255.255.128	192.168.1.129

Step 5: Verify with ping

On PC0, open **Desktop > Command Prompt** and run:

Command	Tests	Expected
`ping 192.168.1.3`	Same lab	Replies
`ping 192.168.1.1`	Own gateway	Replies
`ping 192.168.1.130`	Other lab, through the router	Replies
`ping 192.168.1.131`	Other lab, through the router	Replies

The first ping may show one "Request timed out" while the devices learn each other's addresses; run it again and all four replies come back. If pings to the other lab fail, check that both router ports are On and that each PC's gateway is the router address for its own lab.

Practical 4: Subnet mask and usable hosts program

The program borrows bits for the number of subnets, then prints the mask, network address, usable range, broadcast, and usable hosts for each subnet. It uses no libraries.

```
# subnet.py
base = [192, 168, 1, 0]
prefix = 24
subnets = 2

borrow = 0
while 2 ** borrow < subnets:
    borrow += 1
new_prefix = prefix + borrow

def to_ip(n):
    return ".".join(str((n >> s) & 255) for s in (24, 16, 8, 0))

mask = (0xFFFFFFFF << (32 - new_prefix)) & 0xFFFFFFFF
block = 2 ** (32 - new_prefix)
start = (base[0] << 24) | (base[1] << 16) | (base[2] << 8) | base[3]

print("New prefix   : /" + str(new_prefix))
```

```
print("Subnet mask  :", to_ip(mask))
print("Usable hosts :", block - 2, "per subnet")
print()

for i in range(subnets):
    net = start + i * block
    print("Subnet", i + 1)
    print("  Network   :", to_ip(net) + "/" + str(new_prefix))
    print("  First host:", to_ip(net + 1))
    print("  Last host :", to_ip(net + block - 2))
    print("  Broadcast :", to_ip(net + block - 1))
```

Expected output: prefix `/25`, mask `255.255.255.128`, 126 usable hosts. Subnet 1 is `192.168.1.0/25` (hosts .1 to .126, broadcast .127). Subnet 2 is `192.168.1.128/25` (hosts .129 to .254, broadcast .255).

Formula to remember: usable hosts = 2^(host bits) - 2.

Practical 5: RIP on three branch routers (Packet Tracer)

Layout: R1 - R2 - R3 in a line. Each router has one LAN with a switch and one PC. Use 2911 routers (they have three Gigabit ports) and Cross-Over cables between routers.

Router	Interface	Address	Connects to
R1	Gig0/0	192.168.1.1/24	LAN1
R1	Gig0/1	10.0.12.1/24	R2
R2	Gig0/0	10.0.12.2/24	R1
R2	Gig0/1	10.0.23.1/24	R3
R2	Gig0/2	192.168.2.1/24	LAN2
R3	Gig0/0	10.0.23.2/24	R2
R3	Gig0/1	192.168.3.1/24	LAN3

Steps

1. Place 3 routers, 3 switches, 3 PCs. Wire as in the table (PC to switch to router LAN port; router to router with Cross-Over).

2. Set PCs: PC1 `192.168.1.10` gateway `192.168.1.1`; PC2 `192.168.2.10` gateway `192.168.2.1`; PC3 `192.168.3.10` gateway `192.168.3.1`; all masks `255.255.255.0`.
3. On each router CLI, set the interface addresses and run `no shutdown` on each. Example for R1:

```
enable
configure terminal
interface gigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit
interface gigabitEthernet 0/1
ip address 10.0.12.1 255.255.255.0
no shutdown
exit
```

4. Turn on RIP on each router (advertise its connected networks):

```
! R1
router rip
version 2
no auto-summary
network 192.168.1.0
network 10.0.0.0
end

! R2
router rip
version 2
no auto-summary
network 192.168.2.0
network 10.0.0.0
end

! R3
router rip
version 2
no auto-summary
network 192.168.3.0
network 10.0.0.0
end
```

5. Wait about a minute, then check `show ip route` on R1. You should see `R` entries for 192.168.2.0 and 192.168.3.0.
6. From PC1, run `ping 192.168.2.10` and `ping 192.168.3.10`. Replies confirm RIP is working.

Practical 6: Distance Vector routing simulation

Each router starts knowing only its direct neighbours. It repeatedly checks whether going through a neighbour is cheaper (Bellman-Ford) until no table changes.

Link costs used: R1-R2 = 2, R2-R3 = 3, R1-R3 = 7. The program should discover that R1 reaches R3 more cheaply via R2 (cost 5).

```
# dv.py
INF = 999
names = ["R1", "R2", "R3"]
cost = [[0, 2, 7],
        [2, 0, 3],
        [7, 3, 0]]
n = len(names)

dist = [row[:] for row in cost]
nxt = [[names[j] if i != j and cost[i][j] < INF else "-"
        for j in range(n)] for i in range(n)]

rounds = 0
changed = True
while changed:
    changed = False
    rounds += 1
    for i in range(n):
        for j in range(n):
            for k in range(n):
                if k != i and cost[i][k] < INF and dist[k][j] < INF:
                    new = cost[i][k] + dist[k][j]
                    if new < dist[i][j]:
                        dist[i][j] = new
                        nxt[i][j] = names[k]
                        changed = True
    print("Round", rounds, "done")

print()
```

```
for i in range(n):
    print("Routing table of", names[i])
    print("Dest  Cost  Next hop")
    for j in range(n):
        print(names[j], "  ", dist[i][j], "  ", nxt[i][j])
    print()

def path(src, dst):
    p = [names[src]]
    while src != dst:
        src = names.index(nxt[src][dst])
        p.append(names[src])
    return " -> ".join(p)

print("Shortest path R1 to R3:", path(0, 2), "cost", dist[0][2])
```

Expected result: R1 to R3 is `R1 -> R2 -> R3` with cost 5. To test other networks, change the numbers in `cost`; use `999` where two routers are not directly linked.

Practical 7: TCP client sends a message to the server

The server listens on port 5000. The client connects, sends one message, and the server prints it. Run `server.py` first in one terminal, then `client.py` in another.

server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 5000))
s.listen(1)
print("Server waiting for client...")

conn, addr = s.accept()
print("Connected to", addr)

msg = conn.recv(1024).decode()
print("Message received:", msg)

conn.close()
s.close()
```

client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 5000))

msg = input("Enter message: ")
c.send(msg.encode())

c.close()
```

Flow: socket, bind, listen, accept on the server; socket, connect, send on the client. To use two different computers, replace `127.0.0.1` in the client with the server's IP address.

Practical 8: TCP calculator (client sends two numbers and an operator)

The client sends `number1 number2 operator` as one line. The server calculates and returns the result. Type `*` for multiply and `/` for divide (the server also accepts `x`, `×`, and `÷`).

server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.bind(("127.0.0.1", 6000))
s.listen(1)
print("Calculator server ready...")

conn, addr = s.accept()
data = conn.recv(1024).decode()          # example: "10 5 +"
a, b, op = data.split()
a, b = float(a), float(b)

if op == "+":
    res = a + b
elif op == "-":
    res = a - b
elif op in ("*", "x", "×"):
    res = a * b
```

```
elif op in ("/", "÷"):
    res = a / b if b != 0 else "Error: division by zero"
else:
    res = "Invalid operator"

print("Received:", data, "-> Result:", res)
conn.send(str(res).encode())

conn.close()
s.close()
```

client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
c.connect(("127.0.0.1", 6000))

a = input("Enter first number: ")
b = input("Enter second number: ")
op = input("Enter operator (+ - * /): ")

c.send((a + " " + b + " " + op).encode())
print("Result from server:", c.recv(1024).decode())

c.close()
```

Sample run: inputs `12`, `4`, `/` give `Result from server: 3.0`.

Practical 9: UDP lowercase to uppercase converter

UDP has no connection step: the client just sends a datagram, and the server replies to the address it came from. Run the server first.

server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind(("127.0.0.1", 7000))
print("UDP server ready...")

data, addr = s.recvfrom(1024)
text = data.decode()
print("Received:", text)

s.sendto(text.upper().encode(), addr)
s.close()
```

client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

text = input("Enter a string: ")
c.sendto(text.encode(), ("127.0.0.1", 7000))

data, _ = c.recvfrom(1024)
print("Converted string:", data.decode())

c.close()
```

Sample run: input `hello college` returns `HELLO COLLEGE`.

TCP vs UDP for your viva: TCP needs `listen`, `accept`, and `connect`, and is reliable and ordered. UDP uses `sendto` and `recvfrom`, with no connection and no delivery guarantee.

Practical 10: DHCP, DNS and Web Server for 20 computers

One server runs all three services. A 2960 switch has 24 ports, enough for 20 PCs plus the server.

1. Place one 2960 switch, one **Server-PT**, and 20 PCs. Tip: build one PC, then copy and paste it 19 times. Connect everything to the switch with Straight-Through cables.
2. **Server IP:** click the server > Desktop > IP Configuration. Set `192.168.1.2`, mask `255.255.255.0`, DNS server `192.168.1.2`.

3. **DHCP:** server > Services > DHCP. Set Service **On**, then use the default pool `serverPool` and fill:
   - Default Gateway: `192.168.1.1`
   - DNS Server: `192.168.1.2`
   - Start IP Address: `192.168.1.10`
   - Subnet Mask: `255.255.255.0`
   - Maximum Number of Users: `20`
   - Click **Save**.
4. **DNS:** Services > DNS. Set Service **On**. Name `www.college.edu`, Type **A Record**, Address `192.168.1.2`, click **Add**.
5. **Web server:** Services > HTTP. Set HTTP **On**. Click `index.html` > Edit, change the text to "Welcome to College Website", and Save.
6. **Clients:** on each PC, Desktop > IP Configuration > select **DHCP**. Each PC should show an address from `192.168.1.10` upward and DNS `192.168.1.2`.
7. **Verify:** on any PC, open Desktop > Web Browser, type `http://www.college.edu`, press Go. The welcome page should load. Also try `ping www.college.edu` in Command Prompt.

Practical 11: Email server with two clients

1. Place one switch, one Server-PT, and two PCs. Connect all with Straight-Through cables.
2. **Static IPs:** Server `192.168.1.2`, PC1 `192.168.1.3`, PC2 `192.168.1.4`, all with mask `255.255.255.0`.
3. **Server:** click the server > Services > EMAIL. Turn **SMTP Service** and **POP3 Service** On. Set Domain Name `college.com` and click **Set**.
4. **User accounts:** in the same screen, add user `user1` with password `1234`, then user `user2` with password `1234`. Click **+** after each.
5. **Email client on PC1:** Desktop > Email > Configure Mail. Fill:
   - Your Name: `User One`
   - Email Address: `user1@college.com`
   - Incoming Mail Server: `192.168.1.2`
   - Outgoing Mail Server: `192.168.1.2`
   - User Name: `user1`, Password: `1234`

- Click **Save**.

6. **Email client on PC2:** repeat with `User Two`, `user2@college.com`, user name `user2`, password `1234`.
7. **Send:** on PC1 click **Compose**. To: `user2@college.com`, add a subject and message, click **Send**.
8. **Verify:** on PC2 open Email and click **Receive**. The message from user1 appears in the inbox. Reply to confirm delivery both ways.

Practical 12: Two departments, DHCP, DNS and Web Server

The server sits in Department A. The router carries Department B's DHCP requests to it using a helper address.

Network	Subnet	Router interface
Dept A	192.168.1.0/24	Gig0/0 = 192.168.1.1
Dept B	192.168.2.0/24	Gig0/1 = 192.168.2.1

1. Place one router, two switches, one Server-PT (Dept A), and two PCs per department. Connect router to each switch, and devices to their switches, with Straight-Through cables.
2. **Server IP:** `192.168.1.2`, mask `255.255.255.0`, gateway `192.168.1.1`, DNS `192.168.1.2`.
3. **Router CLI:**

```
enable
configure terminal
interface gigabitEthernet 0/0
ip address 192.168.1.1 255.255.255.0
no shutdown
exit
interface gigabitEthernet 0/1
ip address 192.168.2.1 255.255.255.0
ip helper-address 192.168.1.2
no shutdown
end
write memory
```

4. **DHCP pools** (server > Services > DHCP > On). Edit `serverPool` for Dept A, then add a second pool and click **Add**:

Pool	Gateway	DNS	Start IP	Mask	Max users
deptA	192.168.1.1	192.168.1.2	192.168.1.10	255.255.255.0	50
deptB	192.168.2.1	192.168.1.2	192.168.2.10	255.255.255.0	50

5. **DNS:** Services > DNS On. Add `www.college.edu`, A Record, `192.168.1.2`.
6. **Web:** Services > HTTP On; edit `index.html` if you like.
7. **Test:** set every PC to **DHCP**. Dept A PCs get `192.168.1.x`, Dept B PCs get `192.168.2.x`. From a PC in each department, open the Web Browser and visit `http://www.college.edu`, then ping a PC in the other department.

Practical 13: DHCP, DNS, Web and Email together

This combines Practicals 10 and 11 on one server and one switch. Use the domain `university.edu`.

1. Place one switch, one Server-PT, and three PCs (for example one student, one faculty, one spare). Connect with Straight-Through cables.
2. **Server IP:** `192.168.1.2`, mask `255.255.255.0`, DNS `192.168.1.2`.
3. **DHCP:** Services > DHCP On. Gateway `192.168.1.1`, DNS `192.168.1.2`, Start IP `192.168.1.10`, mask `255.255.255.0`, Max users `50`, Save.
4. **DNS:** Services > DNS On. Add two A Records, both pointing to `192.168.1.2`:
   - `www.university.edu`
   - `mail.university.edu`
5. **Web:** Services > HTTP On. Edit `index.html` with a welcome message.
6. **Email:** Services > EMAIL. Turn SMTP and POP3 On, set Domain Name `university.edu`, click **Set**, then add users `student1` and `faculty1` (password `1234`).
7. **Clients:** set all PCs to **DHCP**. On the student PC, open Desktop > Email > Configure Mail: address `student1@university.edu`, Incoming and Outgoing Mail Server `mail.university.edu`, user name `student1`, password `1234`. Do the same for `faculty1` on the faculty PC.
8. **Verify website:** Web Browser > `http://www.university.edu`.

9. **Verify email:** send from `student1` to `faculty1@university.edu`, then click **Receive** on the faculty PC.

If the email setup fails with the names, enter `192.168.1.2` as the mail server instead; that shows the problem is DNS.

Practical 14: Fixing a website that opens by IP but not by name

Problem: PCs get IP addresses from DHCP but `www.college.edu` does not open. DHCP works, so the fault is in DNS delivery or the DNS records.

Step 1: Build the faulty setup. Use the Practical 10 layout (switch, server `192.168.1.2`, three PCs). Turn DHCP, DNS, and HTTP On, but leave the **DNS Server** box in the DHCP pool empty (or enter a wrong address such as `192.168.1.99`). Add the DNS record `www.college.edu` = `192.168.1.2`.

Step 2: Find the issue.

1. On a PC, open Command Prompt and run `ipconfig /all`. DHCP Server and IP are present, but DNS Server shows `0.0.0.0` (or the wrong address).
2. Run `ping 192.168.1.2`. It works, so the network and web server are fine.
3. Run `ping www.college.edu`. It fails, which confirms name resolution is the problem.

Step 3: Fix it.

1. Server > Services > DHCP: set DNS Server to `192.168.1.2` and click **Save**.
2. Check Services > DNS is **On** and the A record is exactly `www.college.edu` pointing to `192.168.1.2`.
3. On the PC, open IP Configuration, click Static then DHCP again (or run `ipconfig /renew`).
4. `ipconfig /all` now shows DNS `192.168.1.2`.

Step 4: Verify. Open the Web Browser and visit `http://www.college.edu`. The page loads.

Checklist when a site opens by IP but not by name

Symptom	Likely cause	Fix
`ipconfig /all` shows DNS 0.0.0.0	DNS field empty in DHCP pool	Enter the DNS server address in the pool

Symptom	Likely cause	Fix
DNS address is wrong	Typo in the pool or on the PC	Correct it, then renew
Name does not resolve, DNS address right	DNS service Off or no A record	Turn DNS On, add the A record
Name resolves to the wrong place	A record points to the wrong IP	Edit the record to the web server's IP
PC kept an old lease	Lease not renewed	Release and renew, or reset to DHCP

C++ versions of the programs

These are C++ versions of the six programming practicals (2, 4, 6, 7, 8, 9). Practicals 1, 3, 5 and 10 to 14 are Packet Tracer work with no code, so they stay as written above. All eight programs below were compiled with `g++` and tested.

How to compile and run

1. Save each program with the file name shown in its heading.
2. Compile: `g++ hamming.cpp -o hamming`
3. Run: `./hamming` (on Windows: `hamming.exe`)
4. For client-server pairs, compile both files. Open two terminals, run the **server first**, then the client.

The socket programs (7, 8, 9) use Linux/macOS headers. On Windows use WSL, or change three things: include `<winsock2.h>` instead of the Linux headers, call `WSADATA w; WSAStartup(MAKEWORD(2,2), &w);` at the start of `main`, use `closesocket()` instead of `close()`, and compile with `g++ file.cpp -o file -lws2_32`.

Socket functions used in practicals 7, 8 and 9

Function	What it does	Used by
`socket(AF_INET, SOCK_STREAM, 0)`	Creates a TCP socket. `SOCK_DGRAM` makes a UDP socket instead	All
`bind(s, addr, size)`	Attaches the socket to an IP address and port	Servers
`listen(s, 1)`	Marks the socket as waiting for connections	TCP server
`accept(s, ...)`	Waits for a client and returns a new socket for talking to it	TCP server
`connect(s, addr, size)`	Connects to the server's IP and port	TCP client
`send` / `recv`	Send or receive bytes over a TCP connection	TCP
`sendto` / `recvfrom`	Send or receive one datagram, with the other side's address	UDP
`htons(5000)`	Converts a port number to network byte order	All
`inet_addr("127.0.0.1")`	Converts the IP text to a number	All
`close(s)`	Closes the socket	All

`127.0.0.1` means "this same computer". To run on two computers, put the server's real IP address in the client.

C++ Practical 2: Hamming Code (hamming.cpp)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string bits;
    cout << "Enter 4 data bits (example 1011): ";
    cin >> bits;
```

```
    int code[8];                      // positions 1..7 (index 0 unused)
    code[3] = bits[0] - '0';          // d1
    code[5] = bits[1] - '0';          // d2
    code[6] = bits[2] - '0';          // d3
    code[7] = bits[3] - '0';          // d4

    code[1] = code[3] ^ code[5] ^ code[7];   // p1 checks 1,3,5,7
    code[2] = code[3] ^ code[6] ^ code[7];   // p2 checks 2,3,6,7
    code[4] = code[5] ^ code[6] ^ code[7];   // p3 checks 4,5,6,7

    cout << "Transmitted code: ";
    for (int i = 1; i <= 7; i++) cout << code[i];
    cout << endl;

    int pos;
    cout << "Error position to introduce (1-7, 0 for none): ";
    cin >> pos;
    if (pos >= 1 && pos <= 7) code[pos] ^= 1;      // flip one bit

    cout << "Received code   : ";
    for (int i = 1; i <= 7; i++) cout << code[i];
    cout << endl;

    int c1 = code[1] ^ code[3] ^ code[5] ^ code[7];
    int c2 = code[2] ^ code[3] ^ code[6] ^ code[7];
    int c3 = code[4] ^ code[5] ^ code[6] ^ code[7];
    int err = c3 * 4 + c2 * 2 + c1;               // 0 means no error

    if (err == 0) {
        cout << "No error detected" << endl;
    } else {
        cout << "Error detected at position " << err << endl;
        code[err] ^= 1;                           // correct it
        cout << "Corrected code  : ";
        for (int i = 1; i <= 7; i++) cout << code[i];
        cout << endl;
    }

    cout << "Original data   : " << code[3] << code[5] << code[6] << code[7]
<< endl;
    return 0;
}
```

How it works

Part	Explanation
`bits[0] - '0'`	Turns the typed character `'1'` or `'0'` into the number 1 or 0
`code[8]`	Array indexed 1 to 7 so the array index equals the Hamming bit position
Data bits	Placed at positions 3, 5, 6, 7 (the non-powers of 2)
`^` (XOR)	Adds bits without carry; the parity bit makes the group of checked bits have an even number of 1s
p1, p2, p3	Stored at positions 1, 2, 4 (the powers of 2); each checks the positions whose binary form has that bit set
`code[pos] ^= 1`	Flips one bit to simulate a transmission error
`c1, c2, c3`	The receiver repeats the parity checks; a result of 1 means that check failed
`c3 * 4 + c2 * 2 + c1`	Reads the three checks as a binary number, which is the position of the wrong bit

Sample output: input `1011` and error position `5` gives transmitted `0110011`, received `0110111`, "Error detected at position 5", corrected `0110011`, original data `1011`.

C++ Practical 4: Subnet calculator (subnet.cpp)

```
#include <iostream>
#include <string>
using namespace std;

string toIP(unsigned int n) {
    return to_string((n >> 24) & 255) + "." + to_string((n >> 16) & 255) +
"." +
           to_string((n >> 8) & 255) + "." + to_string(n & 255);
}

int main() {
```

```
    unsigned int base = (192u << 24) | (168u << 16) | (1u << 8) | 0u;  //
192.168.1.0
    int prefix = 24;
    int subnets = 2;

    int borrow = 0;
    while ((1 << borrow) < subnets) borrow++;      // bits needed for the
subnets
    int newPrefix = prefix + borrow;

    unsigned int mask = 0xFFFFFFFFu << (32 - newPrefix);
    unsigned int block = 1u << (32 - newPrefix);   // addresses per subnet

    cout << "New prefix   : /" << newPrefix << endl;
    cout << "Subnet mask  : " << toIP(mask) << endl;
    cout << "Usable hosts : " << block - 2 << " per subnet\n\n";

    for (int i = 0; i < subnets; i++) {
        unsigned int net = base + i * block;
        cout << "Subnet " << i + 1 << endl;
        cout << "  Network   : " << toIP(net) << "/" << newPrefix << endl;
        cout << "  First host: " << toIP(net + 1) << endl;
        cout << "  Last host : " << toIP(net + block - 2) << endl;
        cout << "  Broadcast : " << toIP(net + block - 1) << endl;
    }
    return 0;
}
```

How it works

Part	Explanation
`unsigned int base`	The whole IP address as one 32-bit number: each part is shifted left by 24, 16, 8, 0 bits and combined with `|`
`while ((1 << borrow) < subnets)`	Finds how many bits to borrow: 2^borrow must be at least the number of subnets (2 subnets need 1 bit)
`newPrefix`	Old prefix plus borrowed bits, so /24 becomes /25

Part	Explanation
`mask`	Starts with all 32 bits set to 1, then shifts left so only the first `newPrefix` bits stay 1
`block`	Addresses in one subnet: 2^(host bits) = 128 for /25
`block - 2`	Usable hosts: the first address is the network and the last is the broadcast
`toIP()`	Converts the 32-bit number back to dotted text by shifting right and keeping 8 bits at a time (`& 255`)
Loop	Subnet i starts at `base + i * block` ; first host is network + 1; broadcast is network + block - 1

Sample output: `/25` , mask `255.255.255.128` , 126 hosts. Subnet 1 is `192.168.1.0/25` (hosts .1 to .126, broadcast .127). Subnet 2 is `192.168.1.128/25` (hosts .129 to .254, broadcast .255). Change `subnets` to 4 and the program gives /26 with 62 hosts each.

C++ Practical 6: Distance Vector routing (dv.cpp)

```
#include <iostream>
#include <string>
using namespace std;

const int N = 3;
const int INF = 999;

int main() {
    string names[N] = {"R1", "R2", "R3"};
    int cost[N][N] = {{0, 2, 7},
                      {2, 0, 3},
                      {7, 3, 0}};
    int dist[N][N];
    string nxt[N][N];

    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            dist[i][j] = cost[i][j];
            nxt[i][j] = (i != j && cost[i][j] < INF) ? names[j] : "-";
```

```
        }

    bool changed = true;
    int rounds = 0;
    while (changed) {
        changed = false;
        rounds++;
        for (int i = 0; i < N; i++)
            for (int j = 0; j < N; j++)
                for (int k = 0; k < N; k++)
                    if (k != i && cost[i][k] < INF && dist[k][j] < INF &&
                        cost[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = cost[i][k] + dist[k][j];
                        nxt[i][j] = names[k];
                        changed = true;
                    }
        cout << "Round " << rounds << " done\n";
    }

    for (int i = 0; i < N; i++) {
        cout << "\nRouting table of " << names[i] << "\nDest  Cost  Next
hop\n";
        for (int j = 0; j < N; j++)
            cout << names[j] << "    " << dist[i][j] << "     " << nxt[i][j]
<< endl;
    }

    // print the path R1 -> R3
    int src = 0, dst = 2;
    string path = names[src];
    int cur = src;
    while (cur != dst) {
        for (int k = 0; k < N; k++) if (names[k] == nxt[cur][dst]) { cur = k;
break; }
        path += " -> " + names[cur];
    }
    cout << "\nShortest path R1 to R3: " << path << "  cost " << dist[src]
[dst] << endl;
    return 0;
}
```

How it works

Part	Explanation
`cost[N][N]`	Direct link costs between routers. Use `INF` (999) where two routers are not directly connected
`dist[i][j]`	Router i's current best known cost to reach router j; starts equal to `cost`
`nxt[i][j]`	The next router that i should send to in order to reach j
Triple loop	For each router i and destination j, try going through each neighbour k
`cost[i][k] + dist[k][j] < dist[i][j]`	The Bellman-Ford rule: if reaching j via neighbour k is cheaper, update the cost and next hop
`while (changed)`	Repeats the exchange rounds until no table changes, meaning the network has converged
Path printing	Follows the next hops from the source until the destination is reached

Sample output: it converges in 2 rounds. R1 reaches R3 with cost 5 through R2 (cheaper than the direct cost of 7), so the path is `R1 -> R2 -> R3`. To try another network, edit the `cost` matrix and set `N` to the number of routers.

C++ Practical 7: TCP message client and server

tcp_server.cpp

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;

int main() {
    int server = socket(AF_INET, SOCK_STREAM, 0);

    int yes = 1;
```

```
    setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    bind(server, (sockaddr*)&addr, sizeof(addr));
    listen(server, 1);
    cout << "Server waiting for client..." << endl;

    int client = accept(server, nullptr, nullptr);
    cout << "Client connected" << endl;

    char buf[1024];
    memset(buf, 0, sizeof(buf));
    recv(client, buf, sizeof(buf) - 1, 0);
    cout << "Message received: " << buf << endl;

    close(client);
    close(server);
    return 0;
}
```

tcp_client.cpp

```
#include <iostream>
#include <string>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;

int main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    if (connect(sock, (sockaddr*)&addr, sizeof(addr)) < 0) {
        cout << "Could not connect. Is the server running?" << endl;
```

```
        return 1;
    }

    string msg;
    cout << "Enter message: ";
    getline(cin, msg);
    send(sock, msg.c_str(), msg.size(), 0);

    close(sock);
    return 0;
}
```

How it works

Part	Explanation
`sockaddr_in addr{}`	A structure holding the address family (IPv4), port and IP; `{}` sets everything to zero first
`SO_REUSEADDR`	Lets you restart the server straight away without a "port already in use" error
`bind` then `listen`	The server claims port 5000, then starts waiting for connections
`accept`	Blocks until a client connects, then returns a new socket (`client`) used only for that client
`recv(client, buf, ...)`	Reads up to 1023 bytes; we keep the last byte free so the text stays null-terminated
`connect`	The client's request to reach the server; fails if the server is not running
`getline(cin, msg)`	Reads the full line, including spaces
`send`	Sends the message bytes over the connection

Sample output: the server prints "Server waiting for client...", then "Client connected", then "Message received: hello from client".

C++ Practical 8: TCP calculator client and server

The client sends `number1 number2 operator` as one line. The server calculates and sends the answer back.

calc_server.cpp

```
#include <iostream>
#include <sstream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;

int main() {
    int server = socket(AF_INET, SOCK_STREAM, 0);
    int yes = 1;
    setsockopt(server, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(6000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    bind(server, (sockaddr*)&addr, sizeof(addr));
    listen(server, 1);
    cout << "Calculator server ready..." << endl;

    int client = accept(server, nullptr, nullptr);

    char buf[1024];
    memset(buf, 0, sizeof(buf));
    recv(client, buf, sizeof(buf) - 1, 0);      // example: "10 5 +"

    double a, b;
    char op;
    istringstream in(buf);
    in >> a >> b >> op;

    string result;
    if (op == '+')                  result = to_string(a + b);
    else if (op == '-')             result = to_string(a - b);
```

```
    else if (op == '*' || op == 'x') result = to_string(a * b);
    else if (op == '/')              result = (b != 0) ? to_string(a / b) :
"Error: division by zero";
    else                             result = "Invalid operator";

    cout << "Received: " << buf << " -> Result: " << result << endl;
    send(client, result.c_str(), result.size(), 0);

    close(client);
    close(server);
    return 0;
}
```

calc_client.cpp

```
#include <iostream>
#include <string>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;

int main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(6000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    if (connect(sock, (sockaddr*)&addr, sizeof(addr)) < 0) {
        cout << "Could not connect. Is the server running?" << endl;
        return 1;
    }

    string a, b, op;
    cout << "Enter first number: ";  cin >> a;
    cout << "Enter second number: "; cin >> b;
    cout << "Enter operator (+ - * /): "; cin >> op;

    string msg = a + " " + b + " " + op;
    send(sock, msg.c_str(), msg.size(), 0);
```

```
    char buf[1024] = {0};
    recv(sock, buf, sizeof(buf) - 1, 0);
    cout << "Result from server: " << buf << endl;

    close(sock);
    return 0;
}
```

How it works

Part	Explanation
Port 6000	A different port from Practical 7, so both can run without clashing
Client message	Joins the two numbers and the operator into one text line, for example `12 4 /`
`istringstream in(buf)`	Treats the received text like input from the keyboard, so `in >> a >> b >> op` splits it into two numbers and a character
`if / else if` chain	Picks the calculation from the operator; `x` is accepted as multiply too
Division check	Returns an error message instead of crashing when the second number is 0
`to_string`	Converts the numeric answer to text so `send` can transmit it
Client `recv`	Waits for the server's reply and prints it

Sample output: inputs `12`, `4`, `/` give `Result from server: 3.000000`. The server prints `Received: 12 4 / -> Result: 3.000000`.

C++ Practical 9: UDP lowercase to uppercase converter

udp_server.cpp

```
#include <iostream>
#include <cctype>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;

int main() {
    int sock = socket(AF_INET, SOCK_DGRAM, 0);          // SOCK_DGRAM = UDP

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(7000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
    bind(sock, (sockaddr*)&addr, sizeof(addr));
    cout << "UDP server ready..." << endl;

    char buf[1024];
    memset(buf, 0, sizeof(buf));
    sockaddr_in from{};
    socklen_t len = sizeof(from);
    int n = recvfrom(sock, buf, sizeof(buf) - 1, 0, (sockaddr*)&from, &len);
    cout << "Received: " << buf << endl;

    for (int i = 0; i < n; i++) buf[i] = toupper(buf[i]);   // convert to
uppercase

    sendto(sock, buf, n, 0, (sockaddr*)&from, len);         // reply to
sender
    close(sock);
    return 0;
}
```

udp_client.cpp

```
#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
using namespace std;
```

```
int main() {
    int sock = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in server{};
    server.sin_family = AF_INET;
    server.sin_port = htons(7000);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    string text;
    cout << "Enter a string: ";
    getline(cin, text);
    sendto(sock, text.c_str(), text.size(), 0, (sockaddr*)&server,
sizeof(server));

    char buf[1024];
    memset(buf, 0, sizeof(buf));
    recvfrom(sock, buf, sizeof(buf) - 1, 0, nullptr, nullptr);
    cout << "Converted string: " << buf << endl;

    close(sock);
    return 0;
}
```

How it works

Part	Explanation
`SOCK_DGRAM`	Makes a UDP socket: no `listen`, `accept` or `connect` is needed
Server `bind`	Reserves port 7000 so the client's datagram reaches the server
`recvfrom`	Receives one datagram and also fills `from` with the sender's address; returns the number of bytes
`toupper` loop	Converts each received character to uppercase
`sendto(..., from, len)`	Sends the reply to the address the datagram came from, since UDP has no connection
Client `sendto`	Sends straight to the server's IP and port with no connection step

Part	Explanation
Client `recvfrom`	Waits for the reply; passing `nullptr` means we do not need the sender's address

Sample output: input `hello college` returns `HELLO COLLEGE`.

TCP vs UDP for your viva: TCP is connection-based, reliable and ordered (`listen`, `accept`, `connect`, `send`, `recv`). UDP is connectionless and faster, with no delivery guarantee (`sendto`, `recvfrom`).

Easy programs (Python and C++)

These are the simplest versions of the six programs, in Python and C++, written for the exam. They use only basic syntax: variables, `if`, loops and lists. There are no functions or bit tricks. Python and C++ versions give the same output, and all were tested.

How to run

1. Python: `python hamming.py`
2. C++: `g++ hamming.cpp -o hamming` then `./hamming`
3. Socket practicals (7, 8, 9): open two terminals, start the **server** first, then the **client**.
4. On Windows, the C++ socket programs need the small changes listed in the earlier "How to compile and run" part.

Easy Practical 2: Hamming Code

Python (hamming.py)

```
bits = input("Enter 4 data bits (example 1011): ")
d1 = int(bits[0])
d2 = int(bits[1])
d3 = int(bits[2])
d4 = int(bits[3])

# parity bits
p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p3 = d2 ^ d3 ^ d4

# code order: p1 p2 d1 p3 d2 d3 d4
```

```
code = [p1, p2, d1, p3, d2, d3, d4]
print("Sent code    :", code)

pos = int(input("Position to make wrong (1-7, 0 for none): "))
if pos != 0:
    code[pos - 1] = 1 - code[pos - 1]      # flip the bit
print("Received code:", code)

# receiver checks
c1 = code[0] ^ code[2] ^ code[4] ^ code[6]
c2 = code[1] ^ code[2] ^ code[5] ^ code[6]
c3 = code[3] ^ code[4] ^ code[5] ^ code[6]
error = c3 * 4 + c2 * 2 + c1

if error == 0:
    print("No error")
else:
    print("Error at position", error)
    code[error - 1] = 1 - code[error - 1]  # correct it
    print("Corrected code:", code)

print("Data bits    :", code[2], code[4], code[5], code[6])
```

C++ (hamming.cpp)

```
#include <iostream>
using namespace std;

int main() {
    string bits;
    cout << "Enter 4 data bits (example 1011): ";
    cin >> bits;

    int d1 = bits[0] - '0';
    int d2 = bits[1] - '0';
    int d3 = bits[2] - '0';
    int d4 = bits[3] - '0';

    // parity bits
    int p1 = d1 ^ d2 ^ d4;
    int p2 = d1 ^ d3 ^ d4;
    int p3 = d2 ^ d3 ^ d4;
```

```
    // code order: p1 p2 d1 p3 d2 d3 d4
    int code[7] = {p1, p2, d1, p3, d2, d3, d4};

    cout << "Sent code    : ";
    for (int i = 0; i < 7; i++) cout << code[i];
    cout << endl;

    int pos;
    cout << "Position to make wrong (1-7, 0 for none): ";
    cin >> pos;
    if (pos != 0) code[pos - 1] = 1 - code[pos - 1];   // flip the bit

    cout << "Received code: ";
    for (int i = 0; i < 7; i++) cout << code[i];
    cout << endl;

    // receiver checks
    int c1 = code[0] ^ code[2] ^ code[4] ^ code[6];
    int c2 = code[1] ^ code[2] ^ code[5] ^ code[6];
    int c3 = code[3] ^ code[4] ^ code[5] ^ code[6];
    int error = c3 * 4 + c2 * 2 + c1;

    if (error == 0) {
        cout << "No error" << endl;
    } else {
        cout << "Error at position " << error << endl;
        code[error - 1] = 1 - code[error - 1];          // correct it
        cout << "Corrected code: ";
        for (int i = 0; i < 7; i++) cout << code[i];
        cout << endl;
    }

    cout << "Data bits    : " << code[2] << code[4] << code[5] << code[6] <<
endl;
    return 0;
}
```

Step by step

1. Read the 4 data bits and store them as `d1` to `d4`.
2. Work out 3 parity bits with `^` (XOR). Each one covers a different group of data bits.
3. Put all 7 bits in a list in the order `p1 p2 d1 p3 d2 d3 d4`. This is the code that is sent.

4. Type a position (1 to 7) to flip one bit, which acts as the transmission error.
5. The receiver repeats the three checks. Together they make a number, `c3 c2 c1`, in binary.
6. If that number is 0 there is no error. Otherwise it is the position of the wrong bit, so flip it back.

Try it: data `1011`, wrong position `5` gives sent `0110011`, received `0110111`, "Error at position 5", and corrected `0110011`.

Easy Practical 4: Subnet mask and usable hosts

This version works for any network of the form 192.168.1.0/24. Only the last number of the address changes.

Python (subnet.py)

```
prefix = 24        # 192.168.1.0/24
subnets = 2        # how many subnets we want

# how many bits to borrow?
bits = 0
while 2 ** bits < subnets:
    bits = bits + 1

new_prefix = prefix + bits
host_bits = 32 - new_prefix
total = 2 ** host_bits          # addresses in each subnet
usable = total - 2              # remove network and broadcast
mask = 256 - total              # last number of the subnet mask

print("New prefix: /" + str(new_prefix))
print("Subnet mask: 255.255.255." + str(mask))
print("Usable hosts in each subnet:", usable)
print()

for i in range(subnets):
    start = i * total
    print("Subnet", i + 1)
```

```
    print("  Network   : 192.168.1." + str(start))
    print("  First host: 192.168.1." + str(start + 1))
    print("  Last host : 192.168.1." + str(start + total - 2))
    print("  Broadcast : 192.168.1." + str(start + total - 1))
```

C++ (subnet.cpp)

```
#include <iostream>
using namespace std;

int main() {
    int prefix = 24;      // 192.168.1.0/24
    int subnets = 2;      // how many subnets we want

    // how many bits to borrow?
    int bits = 0;
    int power = 1;
    while (power < subnets) {
        power = power * 2;
        bits = bits + 1;
    }

    int newPrefix = prefix + bits;
    int hostBits = 32 - newPrefix;

    int total = 1;                        // addresses in each subnet
    for (int i = 0; i < hostBits; i++) total = total * 2;

    int usable = total - 2;               // remove network and broadcast
    int mask = 256 - total;               // last number of the subnet mask

    cout << "New prefix: /" << newPrefix << endl;
    cout << "Subnet mask: 255.255.255." << mask << endl;
    cout << "Usable hosts in each subnet: " << usable << endl << endl;

    for (int i = 0; i < subnets; i++) {
        int start = i * total;
        cout << "Subnet " << i + 1 << endl;
        cout << "  Network   : 192.168.1." << start << endl;
        cout << "  First host: 192.168.1." << start + 1 << endl;
        cout << "  Last host : 192.168.1." << start + total - 2 << endl;
```

```
        cout << "  Broadcast : 192.168.1." << start + total - 1 << endl;
    }
    return 0;
}
```

Step by step

1. We start with /24 and want 2 subnets.
2. Find how many bits to borrow: keep doubling until we have at least 2 subnets. That is 1 bit.
3. New prefix = 24 + 1 = **/25**. Host bits left = 32 - 25 = 7.
4. Addresses in each subnet = 2^7 = 128. Usable hosts = 128 - 2 = **126**.
5. Subnet mask last number = 256 - 128 = 128, so the mask is `255.255.255.128`.
6. Subnet 1 starts at 0 and subnet 2 starts at 128. In each one the first address is the network, the next is the first host, and the last address is the broadcast.

Try it: the output is `/25`, `255.255.255.128`, 126 hosts. Subnet 1 is `192.168.1.0` (hosts .1 to .126, broadcast .127) and subnet 2 is `192.168.1.128` (hosts .129 to .254, broadcast .255). Change `subnets` to 4 and you get `/26`, `255.255.255.192`, 62 hosts.

Easy Practical 6: Distance Vector routing

Python (dv.py)

```
names = ["R1", "R2", "R3"]

# cost between routers (use 999 if there is no direct link)
cost = [[0, 2, 7],
        [2, 0, 3],
        [7, 3, 0]]

# each router starts knowing only its direct links
dist = [[0, 2, 7],
        [2, 0, 3],
        [7, 3, 0]]

# next router to use for each destination
nxt = [[0, 1, 2],
       [0, 1, 2],
       [0, 1, 2]]
```

```
changed = True
while changed:
    changed = False
    for i in range(3):              # router i
        for j in range(3):          # destination j
            for k in range(3):      # go through neighbour k
                new_cost = cost[i][k] + dist[k][j]
                if new_cost < dist[i][j]:
                    dist[i][j] = new_cost
                    nxt[i][j] = k
                    changed = True

for i in range(3):
    print("Routing table of", names[i])
    for j in range(3):
        print("  to", names[j], "cost", dist[i][j], "next hop", names[nxt[i]
[j]])
    print()

# shortest path from R1 (0) to R3 (2)
start = 0
end = 2
path = names[start]
now = start
while now != end:
    now = nxt[now][end]
    path = path + " -> " + names[now]
print("Shortest path R1 to R3:", path, "| cost =", dist[start][end])
```

C++ (dv.cpp)

```
#include <iostream>
using namespace std;

int main() {
    string names[3] = {"R1", "R2", "R3"};

    // cost between routers (use 999 if there is no direct link)
    int cost[3][3] = {{0, 2, 7},
                      {2, 0, 3},
                      {7, 3, 0}};

    // each router starts knowing only its direct links
```

```
    int dist[3][3] = {{0, 2, 7},
                      {2, 0, 3},
                      {7, 3, 0}};

    // next router to use for each destination
    int nxt[3][3] = {{0, 1, 2},
                     {0, 1, 2},
                     {0, 1, 2}};

    bool changed = true;
    while (changed) {
        changed = false;
        for (int i = 0; i < 3; i++)            // router i
            for (int j = 0; j < 3; j++)        // destination j
                for (int k = 0; k < 3; k++) {  // go through neighbour k
                    int newCost = cost[i][k] + dist[k][j];
                    if (newCost < dist[i][j]) {
                        dist[i][j] = newCost;
                        nxt[i][j] = k;
                        changed = true;
                    }
                }
    }

    for (int i = 0; i < 3; i++) {
        cout << "Routing table of " << names[i] << endl;
        for (int j = 0; j < 3; j++)
            cout << "  to " << names[j] << " cost " << dist[i][j]
                 << " next hop " << names[nxt[i][j]] << endl;
        cout << endl;
    }

    // shortest path from R1 (0) to R3 (2)
    int start = 0, end = 2;
    string path = names[start];
    int now = start;
    while (now != end) {
        now = nxt[now][end];
        path = path + " -> " + names[now];
    }
```

```
    cout << "Shortest path R1 to R3: " << path << " | cost = " << dist[start]
[end] << endl;
    return 0;
}
```

Step by step

1. `cost` holds the direct link costs: R1 to R2 is 2, R2 to R3 is 3, R1 to R3 is 7.
2. `dist` is each router's table. At the start it only knows its direct links.
3. `nxt` remembers which router to send to next for each destination (0 = R1, 1 = R2, 2 = R3).
4. The three loops ask: "for router `i` to reach destination `j`, is going through neighbour `k` cheaper?"
5. If yes, save the new cost and the new next hop. Keep repeating until a full pass changes nothing.
6. Print each router's table, then follow the next hops from R1 to R3 to print the path.

Try it: R1 reaches R3 with cost 5 through R2 (cheaper than the direct cost of 7), so the path is `R1 -> R2 -> R3`. To test another network, change the numbers in `cost` and `dist`; use 999 where two routers have no direct link.

Easy Practical 7: TCP message from client to server

Start the server first, then the client.

Python: tcp_server.py

```
import socket

s = socket.socket()                  # TCP socket
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("127.0.0.1", 5000))          # IP and port
s.listen(1)
print("Server waiting...")

conn, addr = s.accept()              # wait for client
print("Client connected")

message = conn.recv(1024).decode()   # receive message
print("Message received:", message)

conn.close()
s.close()
```

Python: tcp_client.py

```
import socket

c = socket.socket()
c.connect(("127.0.0.1", 5000))       # connect to server

message = input("Enter message: ")
c.send(message.encode())             # send message

c.close()
```

C++: tcp_server.cpp

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int s = socket(AF_INET, SOCK_STREAM, 0);         // TCP socket
    int yes = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    sockaddr_in addr = {};
```

```
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);                    // port
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");  // IP

    bind(s, (sockaddr*)&addr, sizeof(addr));
    listen(s, 1);
    cout << "Server waiting..." << endl;

    int conn = accept(s, NULL, NULL);               // wait for client
    cout << "Client connected" << endl;

    char message[1024] = {0};
    recv(conn, message, 1023, 0);                   // receive message
    cout << "Message received: " << message << endl;

    close(conn);
    close(s);
    return 0;
}
```

C++: tcp_client.cpp

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int c = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(5000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(c, (sockaddr*)&addr, sizeof(addr));     // connect to server

    char message[1024];
    cout << "Enter message: ";
    cin.getline(message, 1024);
    send(c, message, strlen(message), 0);           // send message
```

```
    close(c);
    return 0;
}
```

Step by step

1. **Server:** create a TCP socket, attach it to IP `127.0.0.1` and port 5000 (`bind`), and start waiting (`listen`).
2. **Server:** `accept` pauses until a client connects, then `recv` reads the message and the program prints it.
3. **Client:** create a socket and `connect` to the same IP and port.
4. **Client:** read a message from the keyboard and `send` it.
5. Both sides `close` their sockets.

The `SO_REUSEADDR` line only lets you restart the server straight away; you can ignore it in the viva.

Try it: type `hello from client` in the client. The server prints `Message received: hello from client`.

Easy Practical 8: TCP calculator

The client sends two numbers and an operator (`+ - * /`). The server calculates and sends the result back. Type `*` for multiply and `/` for divide. Start the server first.

Python: calc_server.py

```
import socket

s = socket.socket()
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind(("127.0.0.1", 6000))
s.listen(1)
print("Calculator server ready...")

conn, addr = s.accept()
data = conn.recv(1024).decode()      # example: "12 4 /"
a, b, op = data.split()
a = float(a)
b = float(b)
```

```
if op == "+":
    result = a + b
elif op == "-":
    result = a - b
elif op == "*":
    result = a * b
elif op == "/":
    if b == 0:
        result = "Cannot divide by zero"
    else:
        result = a / b
else:
    result = "Wrong operator"

print("Received:", data, "-> Result:", result)
conn.send(str(result).encode())      # send result back

conn.close()
s.close()
```

Python: calc_client.py

```
import socket

c = socket.socket()
c.connect(("127.0.0.1", 6000))

a = input("Enter first number: ")
b = input("Enter second number: ")
op = input("Enter operator (+ - * /): ")

c.send((a + " " + b + " " + op).encode())      # send all three

result = c.recv(1024).decode()                 # get the answer
print("Result from server:", result)

c.close()
```

C++: calc_server.cpp

```
#include <iostream>
#include <cstdio>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    int yes = 1;
    setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

    sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(6000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    bind(s, (sockaddr*)&addr, sizeof(addr));
    listen(s, 1);
    cout << "Calculator server ready..." << endl;

    int conn = accept(s, NULL, NULL);

    char data[1024] = {0};
    recv(conn, data, 1023, 0);                      // example: "12 4 /"

    double a, b, result = 0;
    char op;
    sscanf(data, "%lf %lf %c", &a, &b, &op);        // split into a, b, op

    char reply[100];
    if (op == '+') {
        result = a + b;
        sprintf(reply, "%g", result);
    } else if (op == '-') {
        result = a - b;
        sprintf(reply, "%g", result);
    } else if (op == '*') {
        result = a * b;
        sprintf(reply, "%g", result);
    } else if (op == '/') {
        if (b == 0) {
            strcpy(reply, "Cannot divide by zero");
```

```
        } else {
            result = a / b;
            sprintf(reply, "%g", result);
        }
    } else {
        strcpy(reply, "Wrong operator");
    }

    cout << "Received: " << data << " -> Result: " << reply << endl;
    send(conn, reply, strlen(reply), 0);             // send result back

    close(conn);
    close(s);
    return 0;
}
```

C++: calc_client.cpp

```
#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int c = socket(AF_INET, SOCK_STREAM, 0);

    sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(6000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(c, (sockaddr*)&addr, sizeof(addr));

    string a, b, op;
    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter second number: ";
    cin >> b;
    cout << "Enter operator (+ - * /): ";
    cin >> op;
```

```
    string message = a + " " + b + " " + op;
    send(c, message.c_str(), message.length(), 0);   // send all three

    char result[1024] = {0};
    recv(c, result, 1023, 0);                        // get the answer
    cout << "Result from server: " << result << endl;

    close(c);
    return 0;
}
```

Step by step

1. **Client:** read the two numbers and the operator, join them into one text like `12 4 /`, and `send` it.
2. **Server:** `accept` the client and `recv` the text.
3. **Server:** split the text into `a`, `b` and `op` (Python `split()`, C++ `sscanf`).
4. **Server:** use `if / elif` to pick `+`, `-`, `*` or `/`. If dividing and `b` is 0, reply with an error message instead.
5. **Server:** `send` the answer back as text.
6. **Client:** `recv` the answer and print it.

Try it: inputs `12`, `4`, `/` give `Result from server: 3` in C++ and `3.0` in Python.

Easy Practical 9: UDP lowercase to uppercase

UDP needs no `listen`, `accept` or `connect`: the client just sends, and the server replies to whoever sent. Start the server first.

Python: udp_server.py

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)    # UDP socket
s.bind(("127.0.0.1", 7000))
print("UDP server ready...")

text, client = s.recvfrom(1024)       # get text and who sent it
text = text.decode()
print("Received:", text)

s.sendto(text.upper().encode(), client)    # send back in capital letters
s.close()
```

Python: udp_client.py

```
import socket

c = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

text = input("Enter a string: ")
c.sendto(text.encode(), ("127.0.0.1", 7000))     # send to server

result, server = c.recvfrom(1024)                # get the reply
print("Converted string:", result.decode())

c.close()
```

C++: udp_server.cpp

```
#include <iostream>
#include <cstring>
#include <cctype>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int s = socket(AF_INET, SOCK_DGRAM, 0);          // UDP socket

    sockaddr_in addr = {};
    addr.sin_family = AF_INET;
    addr.sin_port = htons(7000);
    addr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

```
    bind(s, (sockaddr*)&addr, sizeof(addr));
    cout << "UDP server ready..." << endl;

    char text[1024] = {0};
    sockaddr_in client = {};
    socklen_t size = sizeof(client);
    recvfrom(s, text, 1023, 0, (sockaddr*)&client, &size);   // get text and
who sent it
    cout << "Received: " << text << endl;

    for (int i = 0; text[i] != '\0'; i++)
        text[i] = toupper(text[i]);                  // make capital letters

    sendto(s, text, strlen(text), 0, (sockaddr*)&client, size);  // send back
    close(s);
    return 0;
}
```

C++: udp_client.cpp

```
#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
using namespace std;

int main() {
    int c = socket(AF_INET, SOCK_DGRAM, 0);

    sockaddr_in server = {};
    server.sin_family = AF_INET;
    server.sin_port = htons(7000);
    server.sin_addr.s_addr = inet_addr("127.0.0.1");

    char text[1024];
    cout << "Enter a string: ";
    cin.getline(text, 1024);
    sendto(c, text, strlen(text), 0, (sockaddr*)&server, sizeof(server));
// send

    char result[1024] = {0};
    recvfrom(c, result, 1023, 0, NULL, NULL);        // get the reply
    cout << "Converted string: " << result << endl;
```

```
    close(c);
    return 0;
}
```

Step by step

1. **Server:** create a UDP socket (`SOCK_DGRAM` in C++, `socket.SOCK_DGRAM` in Python) and `bind` it to port 7000.
2. **Server:** `recvfrom` waits for a message and also gives the sender's address.
3. **Server:** change every letter to uppercase (`upper()` in Python, `toupper` in a loop in C++).
4. **Server:** `sendto` the result back to that sender address.
5. **Client:** read a string, `sendto` the server's IP and port, then `recvfrom` the reply and print it.

Try it: input `hello college` gives `Converted string: HELLO COLLEGE`.

TCP vs UDP for the viva: TCP makes a connection first and delivers data reliably and in order. UDP just sends and is faster, with no delivery guarantee.

Packet Tracer practicals: explanation and viva questions

The eight Packet Tracer practicals are 1, 3, 5, 10, 11, 12, 13 and 14. Each one below has a short explanation to say in your own words, then likely viva questions with one or two line answers.

Practical 1: Network topologies

Explanation: A topology is the layout of devices and cables in a network. We built four. Star connects every device to one central switch. Bus connects devices along one shared line (hubs in a chain in Packet Tracer). Mesh connects every device to every other one. Extended star joins several star networks through a central switch. We gave each its own IP range and used `ping` to prove the devices can talk.

Question	Answer
What is a network topology?	The physical or logical arrangement of devices and links in a network.
Why does a star network stop if the central switch fails?	Every device's traffic passes through that one switch, so it is a single point of failure.
How many links does a full mesh of n devices need?	n(n-1)/2. For 4 devices that is 6 links.
Why is bus cheap but unreliable?	It uses one shared cable, but a break splits the network and all devices share the same bandwidth, causing collisions.
What is the difference between a hub and a switch?	A hub sends data out of every port. A switch learns MAC addresses and sends data only to the right port.
When do you use a straight-through cable and a cross-over cable?	Straight-through joins different device types (PC to switch). Cross-over joins the same types (switch to switch).
Which topology is most used today?	Extended star (a hierarchy of switches), because it is easy to expand and a fault stays in one branch.
Which command tests connectivity and how does it work?	`ping`. It sends ICMP echo requests and waits for echo replies.

Practical 3: Subnetting

Explanation: Subnetting splits one network into smaller ones. We split 192.168.1.0/24 into two by borrowing 1 host bit, giving /25 and mask 255.255.255.128. Each subnet has 126 usable hosts. PCs in different subnets are in different networks, so they can only talk through a router. The router's interface in each subnet is that subnet's default gateway.

Question	Answer
What is subnetting and why do we do it?	Dividing a network into smaller networks. It reduces broadcast traffic, improves security and organises the network, for example one subnet per lab.
What is the subnet mask for /25?	255.255.255.128.
How many usable hosts are in a /25?	126, because 2^7 = 128 and we subtract the network and broadcast addresses.
Why subtract 2?	The first address is the network address and the last is the broadcast address; neither can be given to a host.
What is a default gateway?	The router address a PC sends traffic to when the destination is in another network.
Why can't PCs in different subnets talk without a router?	They are in different networks, and moving packets between networks needs a layer 3 device.
What is the network address of 192.168.1.130 with /25?	192.168.1.128.
What does CIDR /25 mean?	The first 25 bits are the network part and the remaining 7 bits are for hosts.
Why does the first ping sometimes time out?	The PC first has to learn the other device's MAC address using ARP; later pings succeed.

Practical 5: RIP routing

Explanation: RIP (Routing Information Protocol) is a distance vector protocol. Each router tells its neighbours which networks it knows and how many hops away they are, and every 30 seconds it repeats this. After a minute all three routers learn all branch networks, so every PC can ping every other PC without us typing static routes. The `network` command tells RIP which directly connected networks to advertise.

Question	Answer
What is RIP?	A distance vector routing protocol that shares routing tables with neighbours.
What metric does RIP use?	Hop count: the number of routers on the way.
What is the maximum hop count?	15. A route with 16 hops is treated as unreachable.
How often does RIP send updates?	Every 30 seconds.
What is the difference between RIPv1 and RIPv2?	v1 is classful and sends no subnet mask. v2 is classless, sends the mask, uses multicast 224.0.0.9 and supports authentication.
What does `no auto-summary` do?	Stops RIP summarising networks to their classful boundary, so subnets are advertised correctly.
Which command shows the routes RIP learned?	`show ip route`. RIP routes are marked with `R`.
What is the difference between distance vector and link state?	Distance vector (RIP) shares its table with neighbours only. Link state (OSPF) gives every router a full map of the network.
What are the problems of RIP?	Slow convergence and the count-to-infinity problem, reduced by split horizon and hold-down timers.
What is the administrative distance of RIP?	120.

Practical 10: DHCP, DNS and Web Server

Explanation: One server gives three services to 20 PCs. DHCP hands each PC an IP address, mask, gateway and DNS address automatically, so nobody types them. DNS turns the name `www.college.edu` into the server's IP address. The web server holds the website. A PC asks DHCP for an address, asks DNS where the site is, then asks the web server for the page.

Question	Answer
What is DHCP?	A service that gives IP configuration to devices automatically.
What are the four steps of DHCP?	DORA: Discover, Offer, Request, Acknowledge.
What does DHCP give a PC?	IP address, subnet mask, default gateway and DNS server address.
What is a lease?	The time a PC may keep the address before it must renew it.
What is DNS and why is it needed?	It translates names to IP addresses, so users remember names instead of numbers.
What is an A record?	A DNS entry that maps a name to an IPv4 address.
Why does the server have a static IP?	Clients and DNS records must always find it at the same address.
What happens if no DHCP server is reachable?	The PC gives itself a 169.254.x.x address (APIPA) and cannot reach other networks.
What is the difference between a web server and a web browser?	The server stores and sends the pages. The browser asks for and displays them.
Which ports do HTTP, DNS and DHCP use?	HTTP 80, DNS 53, DHCP 67 (server) and 68 (client).

Practical 11: Email server

Explanation: The email server has two jobs. SMTP sends mail from a client to the server and between servers. POP3 lets a client download its mail from the server. We create two accounts on the server, set up the email client on each PC with the server address, and send a message from one user to the other. The message goes by SMTP to the server, sits in user2′s mailbox, and user2′s PC collects it by POP3.

Question	Answer
What do SMTP and POP3 do?	SMTP sends email. POP3 receives (downloads) email.
Which ports do they use?	SMTP 25, POP3 110.
What is IMAP and how does it differ from POP3?	IMAP (port 143) keeps mail on the server and syncs across devices. POP3 normally downloads it and removes it from the server.
What is the format of an email address?	username@domain, for example `user1@college.com`.
Why do we set a domain name on the mail server?	The server uses it to know which addresses belong to it.
What is an MX record?	A DNS record that tells which mail server handles mail for a domain.
What is the role of the mail server?	It stores each user's mailbox and forwards messages.
Why do both incoming and outgoing server fields contain the same address here?	One server runs both SMTP and POP3.
What is the difference between a mail client and a mail server?	The client is the user's program for reading and writing mail. The server stores and delivers it.

Practical 12: Two departments with DHCP, DNS and Web

Explanation: Two departments are two separate networks joined by a router. DHCP uses broadcast, and routers do not forward broadcasts, so Department B's PCs cannot reach the server on their own. The `ip helper-address` command on the router passes those requests to the DHCP server. The server has two address pools, one per department, each with its own gateway. DNS and the web server are shared, and traffic between departments goes through the router.

Question	Answer
Why do we need `ip helper-address`?	DHCP requests are broadcasts and routers block broadcasts. The command forwards them to the server as unicast.
What is a broadcast domain?	All devices that receive each other's broadcasts. Each router interface is a separate one.
Why are there two DHCP pools?	Each department is a different subnet and needs its own addresses and gateway.
Why is the gateway different in each pool?	Each department's gateway is the router interface in its own subnet.
Where does the helper address point?	To the DHCP server's IP address (192.168.1.2).
How does a Department B PC open the website?	It sends the request to its gateway, the router forwards it to the server in Department A, and the page returns the same way.
What does DNS give the PCs in both departments?	The same server address, because the DNS server is shared.
How do you check that routing between departments works?	Ping a PC in the other department.

Practical 13: DHCP, DNS, Web and Email together

Explanation: One server runs all four services for the university. DHCP configures the PCs. DNS has two names, `www.university.edu` for the website and `mail.university.edu` for email, both pointing to the server. The web service shows the site. The email service holds the student and faculty mailboxes. The order matters: server IP first, then DHCP, then DNS, then web, then email, then test from the clients.

Question	Answer
Which four services did you configure and what does each do?	DHCP gives addresses, DNS resolves names, HTTP serves the website, SMTP and POP3 handle email.
Why use names like `mail.university.edu` instead of the IP?	Names are easier to remember and keep working if the server's IP changes, since only the DNS record changes.
What does the DNS server need to resolve both names?	Two A records, `www` and `mail`, both pointing to the server.
Can one server run all services?	Yes, since each service listens on a different port. In big networks they are split across servers for load and security.
What if the website works but email fails?	Check the email service is on, the domain name is set, the accounts exist and the client has the correct mail server and user name.
What if everything works by IP but not by name?	The fault is in DNS: the DNS address handed out by DHCP or the DNS records.
How do you verify the whole setup?	Open the website in the browser, then send an email from one user and receive it on the other.

Practical 14: Troubleshooting DNS when DHCP works

Explanation: The PCs got IP addresses, so DHCP works, and a ping to the server's IP works, so the network and web server work. Only the name fails, so the fault is in name resolution. In our setup the DHCP pool had no DNS server address, so the PCs did not know where to ask. We found it with `ipconfig /all`, fixed the DNS field in the pool, renewed the PC's address, and the website opened.

Question	Answer
What is the first thing you check?	`ipconfig /all` on the PC, to see its IP, gateway and DNS server.

Question	Answer
How did you know the problem was DNS and not the network?	`ping` to the server's IP worked but `ping www.college.edu` failed.
What are the possible causes?	DNS address missing or wrong in DHCP, DNS service off, or the A record missing or wrong.
What is the difference between the jobs of DHCP and DNS?	DHCP gives the PC its address and tells it which DNS server to use. DNS turns names into addresses.
How do you make a PC pick up the corrected settings?	Renew the lease: `ipconfig /renew`, or switch the PC to Static and back to DHCP.
Which command tests a DNS name directly?	`nslookup www.college.edu`.
In what order do you troubleshoot a network?	Bottom up: cables and lights, IP address, gateway, DNS, then the application.
What happens if the DNS record points to the wrong IP?	The name resolves, but the browser reaches the wrong machine or fails.

Port numbers to remember

Service	Port	Protocol
HTTP	80	TCP
HTTPS	443	TCP
DNS	53	UDP (and TCP)
DHCP	67 server, 68 client	UDP
SMTP	25	TCP
POP3	110	TCP
IMAP	143	TCP
FTP	21	TCP
Telnet	23	TCP

Service	Port	Protocol
RIP	520	UDP